Sequence -> physicochemical scales -> interpretable features -> explainable ML -> biological mechanism

Interpretable, sequence-based protein intelligence — small-data robust, mechanism-aware, publication-ready.

Every feature is a Part × Split × Scale — every prediction traceable to a residue × property × group comparison.

THE GOLDEN WORKFLOW

canonical pipeline

1 · LOAD	2 · PARTS	3 · FEATURES	4 · MODEL	5 · EXPLAIN
`df_seq, df_scales`	`df_parts`	`df_feat`	`fit`	`visualize`
load_dataset · load_scales	*SequenceFeature.get_df_parts*	*CPP.run*	*TreeModel · dPULearn*	*ShapModel · CPPPlot*

WHICH MODULE SHOULD I USE?

DECISION FLOW

DISCOVERY & FEATURES

Discover discriminative properties between two groups	`CPP`	
Reduce a redundant scale set to medoids	`AAclust`	WRAPPER
Build feature matrix X from features + parts	`SequenceFeature.feature_matrix`	
Drop correlated scales / features	`NumericalFeature.filter_correlation`	
Generate sequence logos	`AAlogo`	

MODELING

Train with positives + unlabeled (no negatives)	`dPULearn`	
Train + RFE + Monte-Carlo importance	`TreeModel`	WRAPPER
Score per-feature SHAP impact	`ShapModel [pro]`	WRAPPER
Score in-silico mutations / design libraries	`AAMut · SeqMut`	

DATA PREP & UTILITIES

Encode sequences (one-hot / integer / window)	`SequencePreprocessor`
Compute pairwise sequence similarity	`comp_seq_sim`
Cluster redundant homologs out of a dataset	`filter_seq [pro]`
Read / write FASTA files	`read_fasta · to_fasta`

FEATURE ONTOLOGY

Feature = Part × Split × Scale

PART	SPLIT	SCALE
where on the sequence	*how to read the part*	*which physicochemical property*
tmd · jmd_n · jmd_c tmd_jmd · jmd_n_tmd_n tmd_c_jmd_c	Segment contiguous Pattern sparse pairs PeriodicPattern i, i+3/4	AAontology (~600 scales) hydrophobicity · charge helix propensity · shape

EXAMPLES

```
TMD × Segment × hydrophobicity      -> membrane insertion
JMD × Pattern × net charge          -> electrostatic recognition
TMD × PeriodicPattern × helix       -> α-helical interface
```

SEQUENCE ANATOMY

TMD model

TMD Target Middle Domain
JMD Juxta Middle Domain (N- and C-terminal flanks)

JMD widths set globally: `aa.options['jmd_n_len'] · ['jmd_c_len']`

dPULearn labels (dpu.labels_):
1: positives · 0: rel-negatives · 2: unlabeled

INSTALL · IMPORT · LOAD

```
# Python >= 3.11
pip install aaanalysis            # core
pip install 'aaanalysis[pro]'     # SHAP, MEME, Bio

import aaanalysis as aa

>>> df_seq    = aa.load_dataset('DOM_GSEC')
>>> df_scales = aa.load_scales()
>>> df_cat    = aa.load_scales(name='scales_cat')
>>> df_feat   = aa.load_features('DOM_GSEC')
>>> df = aa.read_fasta('prots.fa')
```

FEATURE ENGINEERING

```
# parts + matrix
>>> sf = aa.SequenceFeature()
>>> df_parts = sf.get_df_parts(df_seq=df_seq)
>>> X = sf.feature_matrix(
          features=df_feat['feature'],
          df_parts=df_parts,
          df_scales=df_scales)

# correlation filter
>>> nf = aa.NumericalFeature()
>>> X_red = nf.filter_correlation(X=X,
          max_cor=0.7)

# sequence encoding
>>> spp = aa.SequencePreprocessor()
>>> spp.encode_one_hot(seqs)  # (n,L,20)
>>> spp.encode_integer(seqs)  # (n,L)

# redundancy filter (pro)
>>> aa.filter_seq(df_seq, method='cd-hit',
          similarity_threshold=0.7)
```

AAclust

Wrapper · scale reduction

```
>>> aac = aa.AAclust(model_class=KMeans)
>>> aac.fit(X=df_scales.T.values,
          names=list(df_scales),
          n_clusters=100, on_center=True,
          min_th=0.3)
>>> aac.labels_          # cluster id per scale
>>> aac.medoid_names_    # representative scales
>>> aac.centers_         # cluster centers
>>> df_eval = aac.eval(X, list_labels=[aac.labels_])

# matching plot class — same logic, plotting only
>>> acp = aa.AAclustPlot()
>>> acp.centers(X, labels=aac.labels_)
>>> acp.medoids(X, labels=aac.labels_)
```

CPP

flagship · interpretability

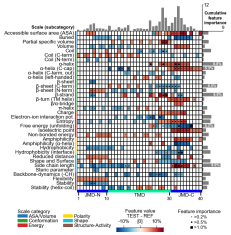
Discover discriminative features between two groups.
Returns a ranked df_feat — the explainability backbone.

```
>>> cpp = aa.CPP(df_scales=df_scales,
          df_parts=df_parts, df_cat=df_cat)
>>> df_feat = cpp.run(labels=labels,
          n_filter=100, max_cor=0.5)

# CPPPlot needs 'feat_importance' col first
>>> tm = aa.TreeModel().fit(X, labels)
>>> df_feat = tm.add_feat_importance(
          df_feat=df_feat)

>>> p = aa.CPPPlot()
>>> p.feature_map(df_feat)   # below
>>> p.profile(df_feat); p.heatmap(df_feat)
```

CPPPlot.feature_map · features × position × scale



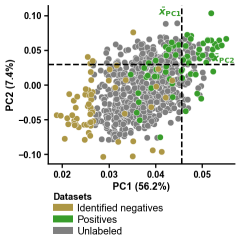
dPULearn

reliable negatives

Find reliable negatives in unlabeled data via deterministic PCA distance.

```
# input labels: 1 = positive, 2 = unlabeled
>>> dpu = aa.dPULearn()
>>> dpu.fit(X=X, labels=labels,
          n_unl_to_neg=n_pos)
>>> dpu.labels_   # 1: pos · 0: rel-neg · 2: unl
>>> dpu.df_pu_    # PCA + selection table
>>> aa.dPULearnPlot.pca(df_pu=dpu.df_pu_,
          labels=dpu.labels_)
```

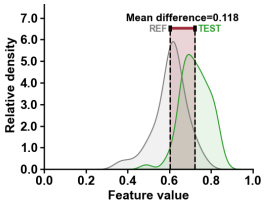
dPULearnPlot.pca · positives + reliable negatives in PC space



Feature distribution

CPPPlot.profile

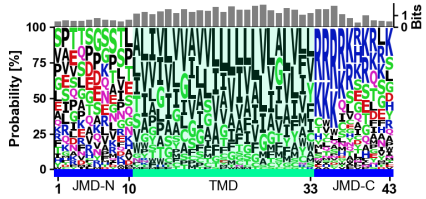
Per-feature KDE: TEST vs REF; mean diff drives ranking.



Sequence logo

AAlogo · AAlogoPlot

Position-specific letter probability across a part.

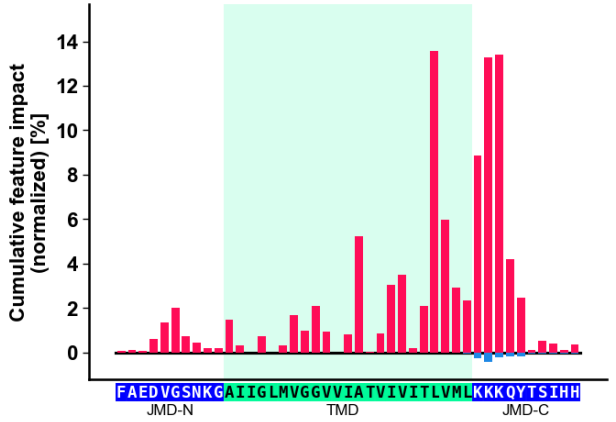


Explainable impact

ShapModel · TreeModel

Cumulative per-residue feature impact (positive vs negative).

```
>>> tm  = aa.TreeModel();  tm.fit(X, labels)
>>> df_feat = tm.add_feat_importance(df_feat=df_feat)
>>> shm = aa.ShapModel(); shm.fit(X, labels)
```



THE 5-MINUTE EXAMPLE

copy-paste this

```
import aaanalysis as aa

# 1 · load
df_seq    = aa.load_dataset('DOM_GSEC')
df_scales = aa.load_scales()

# 2 · parts
sf       = aa.SequenceFeature()
df_parts = sf.get_df_parts(df_seq=df_seq)

# 3 · features (CPP)
cpp = aa.CPP(df_scales=df_scales,
             df_parts=df_parts)
df_feat = cpp.run(labels=df_seq.label)
X = sf.feature_matrix(
      features=df_feat['feature'],
      df_parts=df_parts)

# 4 · model + 5 · explain
tm  = aa.TreeModel();  tm.fit(X, df_seq.label)
df_feat = tm.add_feat_importance(df_feat=df_feat)
shm = aa.ShapModel(); shm.fit(X, df_seq.label)
aa.CPPPlot().feature_map(df_feat)
```

What you get back:

`df_feat`	ranked Part × Split × Scale features
`X`	feature matrix (n_seq × n_feat)
`tm`	fitted classifier with importance_
`shm`	SHAP impact per feature & sample

aa.options · system-level settings

```
# 'off' = use library defaults
aa.options['random_state']         = 42
aa.options['verbose']              = True
aa.options['allow_multiprocessing'] = True

# TMD model (jmd_*_len default None)
aa.options['jmd_n_len'] = 10
aa.options['jmd_c_len'] = 10

# plot labels & system-level scales
aa.options['name_tmd']   = 'TMD'
aa.options['name_jmd_n'] = 'JMD_N'
aa.options['name_jmd_c'] = 'JMD_C'
```

CLASS + PLOT CLASS

mirrored API

*Most analytical classes have a matching *Plot mirror.*

CLASS	PLOT CLASS	KIND
`CPP`	`CPPPlot`	
`AAclust`	`AAclustPlot`	WRAPPER
`AAlogo`	`AAlogoPlot`	
`dPULearn`	`dPULearnPlot`	
`TreeModel`	—	WRAPPER
`ShapModel [pro]`	—	WRAPPER
`AAMut`	`AAMutPlot`	
`SeqMut`	`SeqMutPlot`	
`SequenceFeature`	—	
`NumericalFeature`	—	
`SequencePreprocessor`	—	

WRAPPER sklearn-style — .fit() / .predict() / .eval(), sets *_ attrs

— no plot mirror — pure data manipulation classes

HOW TO CITE

if you use AAanalysis

If you use AAanalysis in your work, please cite the relevant publication. All references include live hyperlinks.

AAclust `[Breimann24a]`

Breimann & Frishman (2024a),
AAclust: k-optimized clustering for selecting redundancy-reduced sets of amino acid scales,
Bioinformatics Advances

AAontology `[Breimann24b]`

Breimann et al. (2024b),
AAontology: An ontology of amino acid scales for interpretable machine learning,
Journal of Molecular Biology

CPP & dPULearn `[Breimann25a]`

Breimann & Kamp et al. (2025),
Charting γ-secretase substrates by explainable AI,
Nature Communications

DESIGN PRINCIPLES

- Explicit over implicit — DataFrames everywhere
- Wrappers (.fit/.predict/.eval) set trailing *_ attrs
- Biological interpretability is first-class

GLOSSARY

vocabulary

TMD	Target Middle Domain — the central segment of interest (e.g. transmembrane stretch, binding region).
JMD	Juxta Middle Domain — the flanks adjoining the TMD; jmd_n on the N-side, jmd_c on the C-side.
Part	Named segment used as feature input: tmd, jmd_n, jmd_c, tmd_jmd, jmd_n_tmd_n, tmd_c_jmd_c.
Split	How a scale is read across a part: Segment (contiguous), Pattern (sparse), PeriodicPattern (i, i+3/4).
Scale	AA -> R mapping. AAontology ships ~600 curated scales organized in two-level categories.
Feature	(Part × Split × Scale) — atomic, residue-grounded, interpretable unit of CPP.
AAontology	Two-level scale taxonomy; CPP uses categories to organize and rank features.
df_seq	Sequence table: entry, sequence, label, TMD bounds (tmd_start, tmd_stop).
df_parts	Wide table — one column per part (tmd, jmd_n, jmd_c, …).
df_feat	Ranked features table: feature, abs_auc, mean_dif, p_val, position, scale, category.
Wrapper	sklearn-style class — .fit / .predict / .eval, sets trailing *_ attrs after fit.
Plot class	*Plot mirror of an analytical class — same arguments, visualization only.
PU labels	Input convention for dPULearn: 1 = positive, 2 = unlabeled. Output: 1 / 0 (rel-neg) / 2.
PU learning	Train with positives + unlabeled — dPULearn finds reliable negatives in the unlabeled pool.
CPP	Comparative Physicochemical Profiling — discovers ranked Part × Split × Scale features.

Layered seeds: seed= (call) > random_state= (init) > aa.options['random_state'] > default.